

# Order-Robust Class Incremental Learning: Graph-Driven Dynamic Similarity Grouping

Guannan Lai<sup>†1</sup>, Yujie Li<sup>†1,2</sup>, Xiangkun Wang<sup>1</sup>, Junbo Zhang<sup>3</sup>, Tianrui Li<sup>4</sup>, and Xin Yang<sup>\*1</sup>

<sup>1</sup>School of Computing and Artificial Intelligence, Southwestern University of Finance and Economics,  
 Email: aignlai@163.com, {liyj1201, xiangkunwang18}@gmail.com, yangxin@swufe.edu.cn

<sup>2</sup>LIACS, Leiden University,

<sup>3</sup>JD Intelligent Cities Research, Email: msjunbozhang@outlook.com

<sup>4</sup>Southwest Jiaotong University, Email: trili@swjtu.edu.cn

## Abstract

*Class Incremental Learning (CIL) aims to enable models to learn new classes sequentially while retaining knowledge of previous ones. Although current methods have alleviated catastrophic forgetting (CF), recent studies highlight that the performance of CIL models is highly sensitive to the order of class arrival, particularly when sequentially introduced classes exhibit high inter-class similarity. To address this critical yet understudied challenge of **class order sensitivity**, we first extend existing CIL frameworks through theoretical analysis, proving that grouping classes with lower pairwise similarity during incremental phases significantly improves model robustness to order variations. Building on this insight, we propose **Graph-Driven Dynamic Similarity Grouping (GDDSG)**, a novel method that employs graph coloring algorithms to dynamically partition classes into similarity-constrained groups. Each group trains an isolated CIL sub-model and constructs meta-features for class group identification. Experimental results demonstrate that our method effectively addresses the issue of class order sensitivity while achieving optimal performance in both model accuracy and anti-forgetting capability. Our code is available at <https://github.com/AIGNLAI/GDDSG>.*

## 1. Introduction

Class Incremental Learning (CIL) necessitates that the model dynamically acquires knowledge of new classes while preserving the knowledge of previously learned

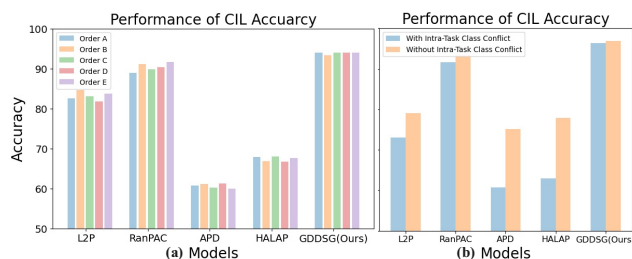


Figure 1. The crucial challenges of CIL (illustration on CIFAR100 dataset). On the left subfigure (a), each model’s performance is shown under varying class orders, testing its robustness to class order sensitivity. On the right subfigure (b), the model’s performance is shown when classes within the same task are similar, evaluating its resilience to intra-task classes with high similarities.

classes within an infinite sequence of tasks [13, 17, 44]. CIL is realistic but a great challenge for deep neural networks [34], where existing works devoted to overcoming catastrophic forgetting (CF) and encouraging knowledge transfer across different tasks [28, 49, 57, 60]. With the rapid advancement of CIL, a growing number of methods [21, 37, 58] have been introduced to address the problem of CF from the perspective of the order in which classes appear (or task order). In practice, the arrival order of each class and the tasks to which they belong are random and the order in which tasks arrive is uncontrollable [3], further resulting in *Class order sensitivity* and *Intra-task class conflicts* [25]. Therefore, designing an order-robust CIL method is essential for the community.

*Class order sensitivity* refers to the model exhibiting significant performance variations depending on the sequence in which classes are introduced [37]. This phenomenon is prevalent in real-world applications (see Figure 1(a)). For

<sup>†</sup> Equal contribution, sorted alphabetically.

<sup>\*</sup> Corresponding author.

instance, in online recommendation systems, the order in which user data classes are received at different time points is difficult to control. If the system initially receives data from relatively few classes, the introduction of subsequent classes may impair the system’s adaptability, resulting in unstable model performance on new tasks. Furthermore, the model’s parameters may be overfitted to the classes of early tasks, diminishing its ability to generalize to subsequent task with new classes. Although existing research, such as APD [58] and HALRP [21], have attempted to mitigate the class order sensitivity problem by modifying network structures, their effectiveness remains limited and has not fundamentally addressed this challenge. Thus, designing a model capable of maintaining stable performance across varying class orders remains a critical unsolved issue in CIL.

*Intra-task class conflicts* refers to the discrepancies in model performance caused by the similarity between classes that are trained simultaneously in a specific task (see Figure 1(b)). In real-world applications, where the arrival of classes in the data stream is uncontrollable, significant similarities among classes can severely impact the model’s resilience. For example, in a specific task from a sequence of tasks, a model may be trained to recognize different breeds within the same species. Within this task, due to the high similarity of features across categories, the model needs to develop resilience in distinguishing between closely related classes. However, existing CIL methods struggle to address this challenge, primarily due to the inherent limitations of the task setting. As CIL incrementally processes different classes, it cannot globally account for all class information, causing class conflicts to accumulate during training and negatively impact model performance. Thus, alleviating class conflicts and improving the model’s generalization ability remains a significant challenge in CIL.

Hence, to tackle the challenges of class order sensitivity and Intra-task class similarity sensitivity, we first conduct an in-depth analysis beyond existing theories. Our theoretical findings suggest that as class similarity decreases in CIL, the model’s robustness to class order increases, which, in turn, mitigates knowledge conflicts both across different tasks and within individual tasks. Then, we propose a similarity graph-based dynamic grouping method, called **Graph-Driven Dynamic Similarity Grouping (GDDSG)**, to maintain the centroids of existing classes and dynamically groups tasks based on class similarity, assigning classes with lower similarity to the same group. This approach innovatively organizes class groups in CIL by utilizing a graph-based technique to minimize inter-group similarity. It dynamically assigns classes based on adaptive similarity thresholds and optimal graph coloring, thereby enhancing model robustness and computational efficiency across tasks. In the incremental learning process, GDDSG continuously updates existing groups or creates new ones,

training a separate model for each group. Consequently, during the prediction phase, decisions are made by aggregating the outputs of multiple models.

Hence, our contributions can be summarised as follows:

- In this paper, we elaborate on existing theories and derive an important Corollary: when the similarity between classes is low, the model’s sensitivity to class order is significantly reduced, leading to a decrease in class conflicts.
- Then, we provide a detailed introduction to the proposed GDDSG method, including its foundational algorithms and basic processes.
- Additionally, we conduct extensive comparative experiments to validate the effectiveness of GDDSG, highlighting its advantages and potential in incremental learning tasks.

## 2. Related Work

**Class-Incremental Learning (CIL)** necessitates a model that can continuously learn new classes while retaining knowledge of previously learned ones [6, 11, 62, 63], which can be roughly divided into several categories. Regularization-based methods incorporate explicit regularization terms into the loss function to balance the weights assigned to new and old tasks [2, 19, 23, 51]. Replay-based methods address the problem of catastrophic forgetting by replaying data from previous classes during the training of new ones. This can be achieved by either directly using complete data from old classes [7, 27, 36, 43] or by generating samples [38, 66], such as employing GANs to synthesize samples from previous classes [9, 26]. Dynamic network methods adapt to new classes by adjusting the network structure, such as adding neurons or layers, to maintain sensitivity to previously learned knowledge while acquiring new tasks. This approach allows the model’s capacity to expand based on task requirements, improving its ability to manage knowledge accumulation in CIL [1, 32, 45, 47]. Recently, CIL methods based on pre-trained models (PTMs) [6, 8, 63] have demonstrated promising results. Prompt-based methods utilize prompt tuning [15] to facilitate lightweight updates to PTMs. By keeping the pre-trained weights frozen, these methods preserve the generalizability of PTMs, thereby mitigating the forgetting in CIL [22, 39, 39, 46, 52, 52, 53]. Model mixture-based methods mitigate forgetting by saving models during training and integrating them through model ensemble or model merge techniques [12, 48, 50, 61, 64, 65]. Prototype-based methods draw from the concept of representation learning [56], leveraging the robust representation capabilities of PTMs for classification with NCM classifiers [29, 33, 62].

**The Order in CIL** remains a significant and unresolved challenge [49]. APD [58] effectively addresses the problem of CF by decomposing model parameters into task-shared and sparse task-specific components, thereby enhancing the

model's robustness to changes in class order. HALRP [21], on the other hand, simulates parameter conversion in continuous tasks by applying low-rank approximation to task-adaptive parameters at each layer of the neural network, thereby improving the model's order robustness. However, the optimization strategies employed by these methods are confined to the network architecture itself and do not fundamentally resolve the underlying issues. Recent theoretical analyses of CIL [3, 25, 37, 55] indicate that prioritizing the learning of tasks (or classes) with lower similarity enhances the model's generalization and resistance to forgetting. Building on these theories, we conducted further research and developed corresponding methods in the following sections.

### 3. Problem Formulation and Theory Analysis

#### 3.1. Problem Formulation

**Definition 1. (Class Incremental Learning (CIL))** Given a sequence of tasks denoted as  $1, \dots, t, \dots$ , each task  $i$  is associated with a training set (i.e., ground-truth data)  $\mathcal{D}^i = \{X^i, Y^i\}$ , where  $X^i$  represents the set of training samples and  $Y^i$  is the set of labels. For task  $i$ , the set of classes is denoted as  $CLS^i$  with the size of  $|CLS^i|$ , representing the number of classes in task  $i$ . With new tasks incrementally appearing, the goal of CIL is to learn a unified model  $\Phi : \mathcal{D}^i \rightarrow \mathbb{R}^d$  mapping input data to an embedding space equipped with a classifier  $f(\cdot)$  that can perform well on all the tasks it has been learned.

Note that for any pair of tasks  $i$  and  $j$  with  $1 \leq i, j \leq n$  and  $i \neq j$ , the sets of classes  $CLS^i$  and  $CLS^j$  are disjoint and data from other tasks is unavailable at the current task, ensuring distinctiveness and non-overlapping nature between classes across each task.

#### 3.2. The Effect of Class Ordering in CIL

In [25], the authors theoretically derived the expected forgetting value and expected generalization error for CIL under a linear model, where  $w_t^*$  denotes the optimal parameters of the model for the  $t$ -th task:

**Theorem 1.** When  $p \geq n + 2$ , we must have:

$$\mathbb{E}[F_T] = \frac{1}{T-1} \sum_{i=1}^{T-1} [(r^T - r^i) \|w_i^*\|^2 + \sum_{j>i}^T c_{i,j} \|w_j^* - w_i^*\|^2] + \frac{p\sigma^2}{p-n-1} (r^i - r^T), \quad (1)$$

$$\mathbb{E}[G_T] = \frac{r^T}{T} \sum_{i=1}^{T-1} \|w_i^*\|^2 + \frac{1-r}{T} \sum_{i=1}^T r^{T-i} \sum_{k=1}^T \|w_k^* - w_i^*\|^2 + \frac{p\sigma^2}{p-n-1} (1 - r^T). \quad (2)$$

where the overparameterization ratio  $r = 1 - \frac{n}{p}$  in this context quantifies the degree of overparameterization in a model, where  $n$  represents the sample size, and  $p$  denotes the number of model parameters [14, 31]. The coefficients  $c_{i,j} = (1-r)(r^{T-i} - r^{j-i} + r^{T-j})$ , with  $1 \leq i < j \leq T$ , correspond to the indices of tasks, and  $\sigma$  denotes a coefficient representing the model's noise level.

**Theorem 1** made a significant contribution to the study of class order in CIL, particularly in the two key expressions:  $\sum_{j>i}^T c_{i,j} \|w_j^* - w_i^*\|^2$  in Equation 1 and  $\sum_{i=1}^T r^{T-i} \sum_{k=1}^T \|w_k^* - w_i^*\|^2$  in Equation 2. These formulas highlight the crucial role that class order plays in CIL. Building on this theory, further in this work, we derive sufficient conditions to ensure order robustness.

**Corollary 1.** A sufficient condition for the reduction of  $\text{Var}(\mathbb{E}[G_T])$  and  $\text{Var}(\mathbb{E}[F_T])$  is that the sum of the squared distances between the optimal parameters of tasks increases, i.e.,  $\sum_{i,j=1}^T \|w_i^* - w_j^*\|^2$  becomes larger.

**Corollary 1** integrates the similarity between tasks with the model's robustness to class order. Through Equation 1 and Equation 2, we observe that both forgetting and generalization errors are influenced by the optimal model gap between any two tasks, represented by  $\|w_i^* - w_j^*\|^2$  for tasks  $i$  and  $j$ . This gap serves as a measure of task similarity: the smaller the gap, the greater the similarity. **Corollary 1** demonstrates that a smaller similarity between tasks enhances the model's robustness in terms of generalization and resistance to forgetting across different class orders. This finding offers valuable insights for the design of new methods.

### 4. The Proposed Method: GDDSG

**Overview.** Figure 2 provides an overview of our proposed method. Using task  $t$  as an example, we begin by projecting all training samples into an embedding space utilizing a pre-trained backbone. In this space, we compute the centroids for each class. Next, we evaluate whether a new centroid  $c_i$  should be integrated into an existing class group  $G_j$ . If  $c_i$  is dissimilar to all classes within  $G_j$ , it is added to the group. If it is similar to any class in an existing group, it remains unassigned. For unassigned centroids, we construct new similarity graphs (SimGraphs) based on their pairwise similarities. We then apply graph coloring theory to these SimGraphs, forming new class groups by clustering dissimilar categories together. Finally, we update the NCM-based classifier with all class groups, facilitating efficient model updates with minimal computational overhead.

#### 4.1. Class Grouping Based on Similarity

**Corollary 1** provides guidance for constructing a sequence of dissimilar tasks. A key idea is to dynamically assign each

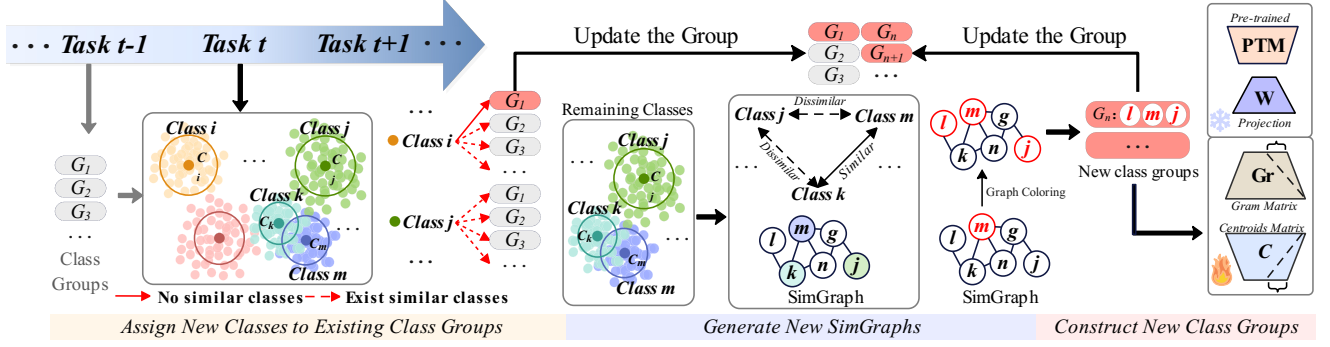


Figure 2. Illustration of The Overall Framework. [best view in color]

new class to a group during CIL, ensuring that the similarity between the new class and other classes within the group is minimized. This approach helps maintain the robustness of each group’s incremental learning process to the order of tasks. For each group, a separate adapter can be trained, and the results from different adapters can be merged during prediction to enhance the model’s overall performance.

In a given CIL task sequence, we organize the classes into several groups. The group list is denoted as  $G = [G_1, \dots, G_k]$ , where each  $G_i$  represents a distinct group of classes. For a specified task  $t$  and each class  $C \in CLS^t$ , our objective is to assign class  $C$  to an optimal group  $G^*$ , ensuring that the new class is dissimilar to all existing classes in that group.

To achieve this objective, we first define the similarity between classes. The similarity between any two classes,  $CLS_i$  and  $CLS_j$ , is determined using an adaptive similarity threshold  $\eta_{i,j}$ . This threshold is computed based on the mean distance between the training samples of each class and their respective centroids in a learned embedding space, as shown below:

$$\eta_{i,j} = \max \left[ \frac{\sum_{k=1}^{|X^t|} \mathbb{I}(y_k^t = i) d(h(x_k^t), \mathbf{c}_i)}{\sum_{k=1}^{|X^t|} \mathbb{I}(y_k^t = i)}, \frac{\sum_{k=1}^{|X^t|} \mathbb{I}(y_k^t = j) d(h(x_k^t), \mathbf{c}_j)}{\sum_{k=1}^{|X^t|} \mathbb{I}(y_k^t = j)} \right], \quad (3)$$

where  $\mathbf{x}^{(t)}$  denotes the  $t$ -th task instance,  $h(\cdot)$  is the feature extraction function defined in Equation 8,  $d: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$  specifies the distance metric space,  $\mathbb{I}(\cdot)$  represents the characteristic function, and the class centroid  $\mathbf{c}_i \in \mathbb{R}^d$  is computed as  $\mathbf{c}_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} \mathbf{x}_j$ .

Building upon this framework, we define the condition under which two classes,  $CLS_i$  and  $CLS_j$ , are considered dissimilar. Specifically, they are deemed dissimilar if the following condition holds:

$$d(\mathbf{c}_i, \mathbf{c}_j) > \eta_{i,j}. \quad (4)$$

Thus, class  $C$  is assigned to group  $G^*$  only if it is dissimilar to all classes within  $G^*$ , and  $G^*$  is the choice with the lowest average similarity:

$$G^* = \arg \min_G \frac{1}{|G|} \sum_{C' \in G} d(C, C'). \quad (5)$$

This approach is consistent with the principles outlined in Corollary 1 and ensures the robustness of the model across the entire task sequence.

## 4.2. Graph-Driven Class Grouping

Graph algorithms provide an efficient method for dynamically grouping classes while minimizing intra-group similarity. In a graph-theoretic framework, classes are represented as nodes, with edge weights quantifying the similarity between them. The flexibility and analytical power of graph structures allow for dynamic adjustment of class assignments in CIL, facilitating optimal grouping in polynomial time. This approach significantly enhances the model’s robustness and adaptability in incremental learning tasks.

Therefore, we can leverage the similarity between classes to construct a SimGraph, defined as follows:

**Definition 2. (SimGraph.)** A SimGraph can be defined as an undirect graph  $SimG = (V, E)$ , where  $V$  is the set of nodes that represent each class’s centroid and  $E$  is the set of edges connecting pair of nodes that represent classes that are determined as similar by Equation 4.

Then, we aim to partition the vertex set of this graph into subsets, with each subset forming a maximal subgraph with no edges between vertices. This problem can be abstracted as the classic NP-hard combinatorial optimization problem of finding a minimum coloring of the graphs. Let  $G^{-1}(\cdot)$  be an assignment of class group identities to each vertex of a graph such that no edge connects two identically labeled vertices (i.e.  $G^{-1}(i) \neq G^{-1}(j)$  for all  $(i, j) \in E$ ). We



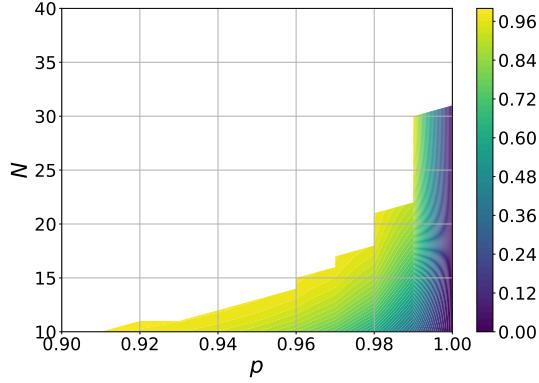


Figure 3. Contour plot delineating the subthreshold region where  $P_{\text{Satisfy Brooks}'} < 0.99$ . The horizontal axis spans  $p \in [0.9, 1.0]$ , representing probability values, while the vertical axis specifies sample sizes  $N \in [10, 40]$ . In regions not displayed, the corresponding  $P_{\text{Satisfy Brooks}'}$  values exceed 0.99.

can formulate the minimum coloring for graph  $\text{Sim}G$  as follows:

$$\mathcal{X}(\text{Sim}G) = \min |\{G^{-1}(k) | k \in V\}|, \quad (6)$$

where  $\mathcal{X}(\text{Sim}G)$  is called the chromatic number of  $\text{Sim}G$  and  $|\cdot|$  denotes the size of the set.

Brooks' theorem [5] offers an upper bound for the graph coloring problem. To apply this in our context, we must demonstrate that the similarity graphs constructed in CIL meet the conditions required by Brooks' theorem. By doing so, we can establish that the problem is solvable and that the solution converges, ensuring the effectiveness of our grouping and class coloring process in class incremental learning. Without loss of generality, we can make the following assumptions:

**Assumption 1.** In the CIL task, class  $C_i$  is randomly sampled without replacement from the set  $\mathcal{U} = \bigcup_{i=1}^{\infty} C_i$ , ensuring that  $C_i \neq C_j$  for all  $i \neq j$ . The probability that any two classes  $C_i$  and  $C_j$  within the set  $\mathcal{U}$  meet the similarity condition (as described in Equation 4) is denoted by  $p$ .

In the CIL scenario with  $N$  classes, the probability of forming an odd cycle is given by  $(p^2(1-p)^{(N-2)})^N = p^{2N}(1-p)^{N^2-2N}$ . Similarly, the probability of forming a complete graph is  $p^{\binom{N}{2}} = p^{\frac{1}{2}N(N-1)}$ . Thus, the probability that the CIL scenario satisfies Brooks' theorem can be expressed as:

$$P_{\text{Satisfy Brooks}'} = 1 - p^{2N}(1-p)^{N^2-2N} - p^{\frac{1}{2}N(N-1)}. \quad (7)$$

Figure 3 illustrates the various values of  $N$  and  $p$  that satisfy Brooks' theorem with a probability of less than 0.99.

Our findings indicate that when  $N > 35$ , the CIL scenario adheres to Brooks' theorem. Furthermore, even with fewer classes, as long as  $p$  does not exceed 0.9, the CIL scenario can still ensure that the similarity graph complies with Brooks' theorem at a confidence level of 0.99. We conclude that class grouping based on the similarity graph is convergent and can be solved efficiently in polynomial time.

For Equation 6, while no algorithm exists that can compute  $\mathcal{X}(\text{Sim}G)$  in polynomial time for all cases, efficient algorithms have been developed that can handle most problems involving small to medium-sized graphs, particularly the similarity graph  $\text{Sim}G$  discussed here. In practical scenarios, such graphs are typically sparse. Notably, in conjunction with the above analysis, the similarity graph  $\text{Sim}G$  in the CIL scenario satisfies the non-odd cycle assumption in Brooks' theorem [5]. For non-complete similarity graphs  $\text{Sim}G$ , we have  $\mathcal{X}(\text{Sim}G) \leq \Delta(\text{Sim}G)$ , where  $\Delta(\text{Sim}G)$  represents the maximum vertex degree in  $\text{Sim}G$ .

Therefore, we can apply a simple yet effective greedy method, the Welsh-Powell graph coloring algorithm [54]. This algorithm first sorts all nodes in the graph in descending order based on their degree and then assigns a color to each node, prioritizing those with higher degrees. During the coloring process, the algorithm selects the minimum available color for each node that differs from its neighbors, creating new color classes when necessary. The time complexity of this algorithm is  $O(|V|^2)$ , primarily due to the color conflict check between each node and its neighbors. In theory, the maximum number of groupings produced by this algorithm is  $\max_{i=1}^n \min\{\deg(v'_i) + 1, i\}$ , with an error margin of no more than 1, where  $V'$  is the sequence of nodes sorted by degree, derived from  $V$ .

### 4.3. Overall Process

**Training Pipeline.** Building upon the theoretical foundations in Section 3.1, we now formalize the complete training procedure. Our framework leverages a frozen pre-trained feature extractor  $\phi(\cdot)$ , augmented with trainable random projections  $W \in \mathbb{R}^{L \times M}$  where  $M \gg L$ , to enhance representation capacity. For each input  $x_i^t$  from class group  $s$ , we compute its expanded feature:

$$h(x_i^t) = g(\phi(x_i^t)W) \in \mathbb{R}^M, \quad (8)$$

where  $g(\cdot)$  denotes the nonlinear activation.

The core learning paradigm reframes classification as regularized least-squares regression. Let  $H_s^t \in \mathbb{R}^{N_s^t \times M}$  be the feature matrix and  $Y_s^t \in \mathbb{R}^{N_s^t \times L_s^t}$  denote the one-hot label matrix for class group  $s$ . We optimize the projection matrix  $\Theta_s^t \in \mathbb{R}^{M \times L_s^t}$  through:

$$\min_{\Theta} \|Y_s^t - H_s^t \Theta_s^t\|_F^2 + \lambda \|\Theta_s^t\|_F^2, \quad (9)$$

Table 1. Results (%) of CL Methods on Both Fine-grained Datasets and General Vision Dataset (Split into 10 Tasks). Among Them, The Best Results Are Bolded for Emphasis, While The Second-best Results Are Underlined.

| Method       | CIFAR100                           |                                   | CUB200                             |                                   | Dog                                |                                   | OB                                 |                                   |
|--------------|------------------------------------|-----------------------------------|------------------------------------|-----------------------------------|------------------------------------|-----------------------------------|------------------------------------|-----------------------------------|
|              | $A_N$ ( $\uparrow$ )               | $F_N$ ( $\downarrow$ )            | $A_N$ ( $\uparrow$ )               | $F_N$ ( $\downarrow$ )            | $A_N$ ( $\uparrow$ )               | $F_N$ ( $\downarrow$ )            | $A_N$ ( $\uparrow$ )               | $F_N$ ( $\downarrow$ )            |
| Finetune     | 67.86 $\pm$ 0.46                   | 31.25 $\pm$ 1.76                  | 48.91 $\pm$ 0.49                   | 47.68 $\pm$ 3.48                  | 45.64 $\pm$ 4.40                   | 46.19 $\pm$ 3.68                  | 61.73 $\pm$ 5.70                   | 31.84 $\pm$ 4.87                  |
| L2P          | 83.45 $\pm$ 0.25                   | 8.50 $\pm$ 0.43                   | 66.57 $\pm$ 1.37                   | 12.00 $\pm$ 1.41                  | 65.09 $\pm$ 3.74                   | 9.44 $\pm$ 2.93                   | 70.10 $\pm$ 8.86                   | 13.70 $\pm$ 3.39                  |
| DualPrompt   | 82.74 $\pm$ 0.85                   | 7.19 $\pm$ 1.18                   | 68.21 $\pm$ 1.55                   | 12.08 $\pm$ 4.65                  | 70.90 $\pm$ 0.57                   | 10.68 $\pm$ 1.35                  | 69.73 $\pm$ 8.26                   | 11.91 $\pm$ 2.28                  |
| CODA-Prompt  | 86.86 $\pm$ 4.26                   | 6.04 $\pm$ 0.71                   | 73.91 $\pm$ 1.45                   | 7.84 $\pm$ 0.10                   | 74.09 $\pm$ 0.69                   | 10.05 $\pm$ 0.07                  | 77.59 $\pm$ 8.41                   | 9.05 $\pm$ 1.89                   |
| SimpleCIL    | 77.63 $\pm$ 2.01                   | 7.21 $\pm$ 0.48                   | 83.32 $\pm$ 2.03                   | 5.89 $\pm$ 1.26                   | 83.23 $\pm$ 0.72                   | 5.86 $\pm$ 0.43                   | 72.15 $\pm$ 0.02                   | 8.31 $\pm$ 0.31                   |
| ADAM         | 84.15 $\pm$ 0.77                   | 5.03 $\pm$ 0.15                   | 83.89 $\pm$ 2.07                   | 5.86 $\pm$ 1.12                   | 83.98 $\pm$ 0.34                   | 6.08 $\pm$ 0.54                   | 72.84 $\pm$ 0.29                   | 8.23 $\pm$ 0.16                   |
| EASE         | 86.95 $\pm$ 0.41                   | 6.74 $\pm$ 0.43                   | 85.78 $\pm$ 4.22                   | 6.27 $\pm$ 2.98                   | 81.03 $\pm$ 1.38                   | 8.18 $\pm$ 0.50                   | 74.56 $\pm$ 4.32                   | 8.60 $\pm$ 1.40                   |
| RanPAC       | 90.50 $\pm$ 0.34                   | 3.49 $\pm$ 0.16                   | 89.23 $\pm$ 0.36                   | 3.91 $\pm$ 0.45                   | 85.37 $\pm$ 0.41                   | 5.78 $\pm$ 0.42                   | 78.86 $\pm$ 0.16                   | 7.62 $\pm$ 0.18                   |
| GDDSG (Ours) | <b>94.00 <math>\pm</math> 0.03</b> | <b>0.78 <math>\pm</math> 0.09</b> | <b>91.64 <math>\pm</math> 1.86</b> | <b>1.92 <math>\pm</math> 1.29</b> | <b>92.64 <math>\pm</math> 0.48</b> | <b>1.42 <math>\pm</math> 0.08</b> | <b>87.33 <math>\pm</math> 0.32</b> | <b>0.96 <math>\pm</math> 0.16</b> |

where  $\lambda$  controls regularization strength. The closed-form solution is:

$$\Theta_s^t = (H_s^{t\top} H_s^t + \lambda I)^{-1} H_s^{t\top} Y_s^t. \quad (10)$$

For incremental updates, we maintain two key components: the Gram matrix  $Gram_s^t$  capturing feature correlations, and the prototype matrix  $C_s^t$  encoding class centroids. When new task  $t$  arrives with  $N_s^t$  samples:

$$Gram_s^t = Gram_s^{t-1} + \sum_{n=1}^{N_s^t} h(x_i^t)^\top h(x_i^t), \quad (11)$$

$$C_s^t = \begin{bmatrix} C_s^{t-1} & \underbrace{\mathbf{0}_M \mathbf{0}_M \cdots \mathbf{0}_M}_{(L_s^t - L_s^{t-1}) \text{ times}} \end{bmatrix} + \sum_{n=1}^{N_s^t} h(x_i^t)^\top y(x_i^t). \quad (12)$$

The regularization parameter  $\lambda$  is adaptively selected from a candidate pool  $\Lambda$  through cross-validation on a held-out calibration set, minimizing the empirical risk:

$$\lambda^* = \arg \min_{\lambda \in \Lambda} \|Y_{\text{val}} - H_{\text{val}}(Gram_{\text{val}} + \lambda I)^{-1} C_{\text{val}}\|_F^2. \quad (13)$$

Additionally, group descriptors are constructed through prototype similarity analysis. For each training instance  $(x, y) \in \mathcal{D}^t$ , we generate meta-features dataset as:

$$\mathcal{D}_g = \{(\rho(x), G^{-1}(y))\}_{(x,y) \in \mathcal{D}^t}, \quad (14)$$

where  $\rho(x) = [d(h(x), \mathbf{c}_1), \dots, d(h(x), \mathbf{c}_k)]^\top$  denotes the concatenated distance vector measuring similarity between the sample embedding and prototype vectors.

**Inference Pipeline.** Given test sample  $x^*$ , its group identification can be learned via  $\hat{g} = \mathcal{M}_g(\rho(x^*))$ , where  $\mathcal{M}_g$  is the class group predict model trained with  $\mathcal{D}_g$ . Then, the prediction will be performed within the selected group via  $\hat{y} = \arg \max_{c \in \mathcal{C}_g} (g(\phi(x^*)W)(Gram_{\hat{g}} + \lambda I)^{-1} C_{\hat{g}}[:, c])$ .

## 5. Experiment

### 5.1. Experimental Setup

**Datasets.** Since most pre-trained models are currently trained on ImageNet-21K [10], we aim to assess the model's performance on entirely new data. To demonstrate the robustness of our model to task similarity, we conduct experiments using several datasets, including CIFAR100 [20], CUB200 [42], Stanford Dogs [18], and OmniBenchmark (OB) [59]. These datasets are divided into multiple, equally sized tasks, and various class orders are tested to evaluate the model's performance across different orders.

**Baseline.** For fairness, we only compare against CL methods that have utilized pre-trained models in recent years. We compare GDDSG with the following six latest and effective CL methods with the PILOT toolbox [40]: L2P [53], Dualprompt [52], CODA-Prompt [39], SimpleCIL [62], ADAM [65], EASE [64], RanPAC [29].

**Implementations.** Our code, implemented in PyTorch, has been open-sourced for accessibility. All experiments were conducted on a single Nvidia RTX 3090 GPU, using three random seeds, 1993, 2024, and 4202, to compute the average for a more robust model evaluation. We use a ViT-B/16 model, which is self-supervised and pre-trained on ImageNet-21K.  $\mathcal{M}_g$  is a soft voting model consisting of RandomForest [4], KNN [35], and LightGBM [16].

**Metrics.** We employ average final accuracy  $A_N$  and average forgetting rate  $F_N$  as metrics [53].  $A_N$  is the average final accuracy concerning all past classes over  $N$  tasks.  $F_N$  measures the performance drop across  $N$  tasks, offering valuable information about plasticity and stability during CL. Following the protocol in [24], we use the Order-normalized Performance Disparity (OPD) metric to assess the robustness of the class order. OPD is calculated as the performance difference of task  $t$  across  $R$  random class orders, defined as:

$$\text{OPD}_t = \max\{\bar{A}_t^1, \dots, \bar{A}_t^R\} - \min\{\bar{A}_t^1, \dots, \bar{A}_t^R\}. \quad (15)$$

To characterize both extreme scenarios and over-

Table 2. Ablation Experiment.

| $A_N$                          | CIFAR100     | CUB200       | Dog          | OB           |
|--------------------------------|--------------|--------------|--------------|--------------|
| w/o Class Groups               | 74.32        | 72.86        | 69.49        | 66.85        |
| w/o SimGraphs and Class Groups | 89.96        | 87.32        | 83.12        | 74.21        |
| GDDSG                          | <b>93.99</b> | <b>92.95</b> | <b>92.30</b> | <b>87.56</b> |
| $F_N$                          | CIFAR100     | CUB200       | Dog          | OB           |
| w/o Class Groups               | 12.42        | 16.70        | 14.13        | 20.95        |
| w/o SimGraphs and Class Groups | 4.12         | 3.78         | 5.99         | 9.46         |
| GDDSG                          | <b>0.72</b>  | <b>1.01</b>  | <b>1.48</b>  | <b>1.07</b>  |

all performance, we introduce two consolidated metrics: Maximum OPD (MOPD) defined as  $MOPD = \max \{OPD_1, \dots, OPD_T\}$ , and Average OPD (AOPD) calculated by  $AOPD = \frac{1}{T} \sum_{t=1}^T OPD_t$ .

## 5.2. Experimental Results

**Main Results.** Table 1 highlights the strong performance of our proposed GDDSG method in terms of accuracy and resistance to forgetting. The results demonstrate that GDDSG consistently outperforms other techniques, achieving state-of-the-art (SOTA) performance. Notably, GDDSG shows marked improvements in both accuracy and forgetting rate. Compared to the previous SOTA method, RanPAC, our approach achieves significantly higher accuracy while maintaining a low forgetting rate of around 1%, underscoring GDDSG’s superior effectiveness in the CIL environment.

**Ablation analysis.** Our method’s two components, SimGraphs and Class Groups, operate as a unified whole. Only after generating the SimGraphs can construct the Class Groups. Therefore, we can only conduct ablation experiments on either individual Class Groups or the SimGraphs and Class Groups combination as a whole, with results shown in Table 2. The results demonstrate a significant decrease in model performance after conducting the ablation, validating the effectiveness of SimGraphs and Class Groups.

**Robustness to Class Order.** We conducted comparative experiments on existing order-robust CIL methods, including APD [24], APDfix [24], and HALRP [21], using 10 different class orders across four datasets and calculating their MOPD and AOPD metrics. The experimental results, presented in Figure 4, show that our proposed GDDSG method demonstrates excellent robustness to category order. MOPD decreased across all datasets, with AOPD showing a significant reduction, underscoring the practical effectiveness of our approach.

**Analysis of Class Group Counts.** Figure 5 illustrates how the number of class groups changes as tasks arrive and task lengths vary during the execution of the GDDSG algorithm.

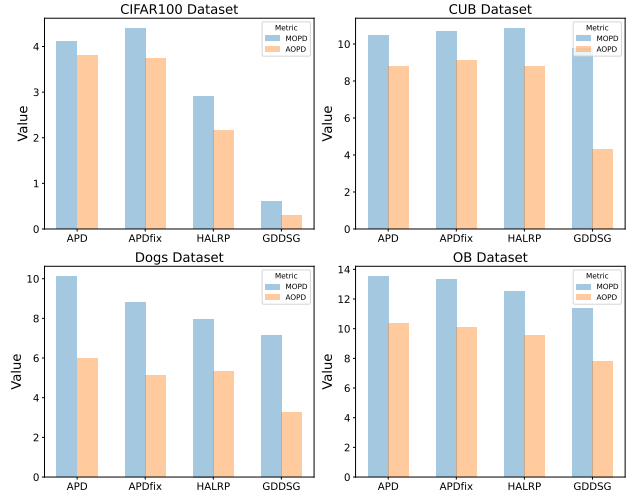


Figure 4. Robustness of Different Methods to Class Order: MOPD (Blue) and AOPD (Orange) Indicators, with GDDSG Performing Well Across Four Datasets

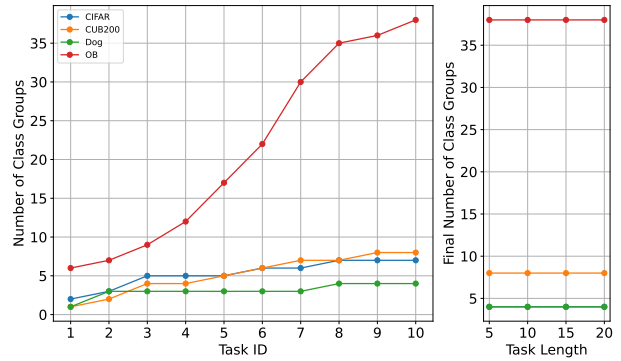


Figure 5. Analysis of Class Group Counts: The Left Figure Shows Changes in Class Group Counts as the Number of Tasks Increases, and the Right Figure Shows Changes as Task Length Varies.

We observe that for relatively homogeneous datasets, such as CIFAR-100 and CUB-200, the optimal number of class groups generated remains relatively low and tends to stabilize midway through the task sequence. In contrast, datasets with broader domains and more categories, such as OB, result in a higher number of optimal class groups. Despite this increase, GDDSG maintains accurate matching, demonstrating its strong generalization capability.

Additionally, we simulate varying frequencies of intra-task class conflicts by altering the number of categories within a single task, which leads to differences in both intra-task and inter-task similarities. The results indicate that the optimal number of class groups determined by the GDDSG algorithm consistently converges to a stable value. This demonstrates that, for a specific dataset and pre-trained

Table 3. The Class Group Number And Final Results of GDDSG Under Different Pre-trained Backbones.

| Dataset  | Metric                         | Backbone     |           |             |
|----------|--------------------------------|--------------|-----------|-------------|
|          |                                | ViT B/16     | ResNet-50 | ResNet-18   |
| CIFAR100 | $\mathcal{X}$ ( $\downarrow$ ) | <b>7</b>     | 39        | 34          |
|          | $A_N$ ( $\uparrow$ )           | <b>93.99</b> | 83.25     | 80.97       |
|          | $F_N$ ( $\downarrow$ )         | 0.72         | 0.73      | <b>0.63</b> |
| CUB200   | $\mathcal{X}$ ( $\downarrow$ ) | <b>8</b>     | 34        | 42          |
|          | $A_N$ ( $\uparrow$ )           | <b>92.95</b> | 70.83     | 51.44       |
|          | $F_N$ ( $\downarrow$ )         | <b>1.01</b>  | 2.43      | 18.32       |
| Dog      | $\mathcal{X}$ ( $\downarrow$ ) | 4            | <b>3</b>  | 18          |
|          | $A_N$ ( $\uparrow$ )           | <b>92.30</b> | 82.65     | 70.56       |
|          | $F_N$ ( $\downarrow$ )         | <b>1.48</b>  | 1.74      | 5.34        |
| OB       | $\mathcal{X}$ ( $\downarrow$ ) | 38           | <b>36</b> | 44          |
|          | $A_N$ ( $\uparrow$ )           | <b>87.56</b> | 77.24     | 62.35       |
|          | $F_N$ ( $\downarrow$ )         | <b>1.07</b>  | 1.17      | 2.14        |

model, the optimal number of class groups is determined solely by the dataset itself, independent of factors such as task length and order in the CIL environment. This stability arises because the GDDSG algorithm primarily relies on data similarity, while disregarding task-specific information in real-world scenarios. The robustness of this approach is theoretically supported by Brooks’ theorem [5] and the Welsh–Powell algorithm [54]. In conclusion, the GDDSG algorithm exhibits strong robustness to variations in task similarity, length, and order, making it highly valuable for a wide range of applications.

**Detailed Analysis of Backbone.** Table 3 presents the optimal number of class groups that GDDSG generates under various pre-trained backbone networks, along with the corresponding  $A_N$  and  $F_N$  values. The results indicate that smaller backbones, such as ResNet18 and ResNet50, yield reduced accuracy and higher forgetting rates compared to using ViT as the backbone. Nonetheless, performance with ResNet50 remains highly competitive, achieving accuracy comparable to L2P while maintaining a relatively low forgetting rate. This further highlights the robustness of GDDSG, as it can reach performance levels similar to those of richer, more powerful backbones, even when using networks with fewer parameters and lower representational capacity.

Another noteworthy observation is that, for the same dataset, the number of class groups varies depending on the backbone network. Specifically, when using ResNet18 or ResNet50, the number of class groups for CIFAR-100 and CUB-200 increases significantly, whereas it decreases for Stanford Dogs and OB. This suggests that, for certain datasets, backbones with lower representational power may erroneously classify some originally dissimilar classes as similar, leading to an increase in class groups. In contrast, other datasets remain unaffected by the choice of back-

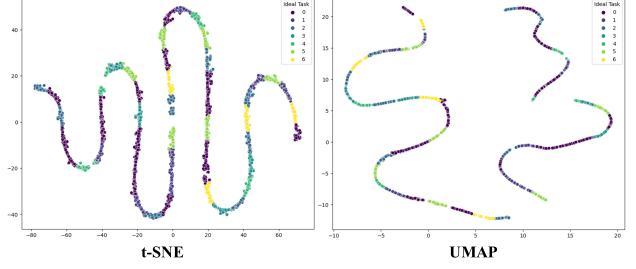


Figure 6. Visualization of The Distance Feature via T-SNE and UMAP After Training on All Tasks (Split CIFAR100 Dataset).

bone. This highlights that the impact of the backbone on the GDDSG algorithm is highly dataset-dependent.

**Detailed Analysis of Class Group Identification.** To better understand the mechanism of class group identification, we visualized  $\mathcal{D}_g$  using t-SNE [41] and UMAP [30] on the Split CIFAR100 dataset. Figure 6 presents the visualized results. The observations reveal that the distance features essentially conform to piecewise functions in high dimensions, exhibiting strong linear separability and powerful representation capabilities. Consequently, a class group identification matching model can be effectively fitted using some classical machine learning models, enabling fairly accurate predictions. However, it is crucial to emphasize the importance of this step for GDDSG. Under the partition of the GDDSG algorithm, the accuracy of a single class group can reach nearly 100%. Therefore, the precision of class group matching directly determines the overall model’s accuracy.

## 6. Conclusion & Limitation

This study addresses the critical challenge of class order sensitivity in Class Incremental Learning (CIL), where model performance significantly degrades under varying class arrival sequences. By introducing GDDSG, a graph-driven framework that dynamically partitions classes into similarity-constrained groups and coordinates isolated sub-models with joint prediction, we theoretically and empirically mitigate the impact of class sequence variations. Experiments validate that our method not only reduces sensitivity to class order but also achieves state-of-the-art accuracy and anti-forgetting performance. This work provides a benchmark for developing robust CIL methods with dynamic data streams.

Inevitably, our method has certain limitations. First, GDDSG currently relies on NCM classifiers. In future work, we aim to explore order-robust CIL approaches with Softmax strategies. Also, while the memory overhead remains small, it could be further streamlined for efficiency, and we intend to address this limitation with future studies.



## Acknowledgement

This work was supported by the National Natural Science Foundation of China (No. 62476228, No. 72242106), the Sichuan Science and Technology Program (No. 2024ZYD0180), and the Major Basic Research Project of Shandong Provincial Natural Science Foundation (No. ZR2024ZD03).

## References

- [1] Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. Expert gate: Lifelong learning with a network of experts. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3366–3375, 2017. 2
- [2] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European Conference on Computer Vision*, pages 139–154, 2018. 2
- [3] Samuel J Bell and Neil D Lawrence. The effect of task ordering in continual learning. *arXiv preprint arXiv:2205.13323*, 2022. 1, 3
- [4] Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001. 6
- [5] Rowland Leonard Brooks. On colouring the nodes of a network. In *Mathematical Proceedings of the Cambridge Philosophical Society*, pages 194–197. Cambridge University Press, 1941. 5, 8
- [6] Boxi Cao, Qiaoyu Tang, Hongyu Lin, Xianpei Han, Jiawei Chen, Tianshu Wang, and Le Sun. Retentive or forgetful? diving into the knowledge memorizing mechanism of language models. *arXiv preprint arXiv:2305.09144*, 2023. 2
- [7] Hyuntak Cha, Jaeho Lee, and Jinwoo Shin. Co2l: Contrastive continual learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9516–9525, 2021. 2
- [8] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. Adaptformer: Adapting vision transformers for scalable visual recognition. *Advances in Neural Information Processing Systems*, 35:16664–16678, 2022. 2
- [9] Yulai Cong, Miaoyun Zhao, Jianqiao Li, Sijia Wang, and Lawrence Carin. Gan memory with no forgetting. *Advances in Neural Information Processing Systems*, 33:16481–16494, 2020. 2
- [10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255. Ieee, 2009. 6
- [11] Shibhansh Dohare, J Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A Rupam Mahmood, and Richard S Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, 2024. 2
- [12] Qiankun Gao, Chen Zhao, Yifan Sun, Teng Xi, Gang Zhang, Bernard Ghanem, and Jian Zhang. A unified continual learning framework with general parameter-efficient tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11483–11493, 2023. 2
- [13] Dipam Goswami, Yuyang Liu, Bartłomiej Twardowski, and Joost van de Weijer. Fecam: Exploiting the heterogeneity of class distributions in exemplar-free continual learning. In *Advances in Neural Information Processing Systems*, pages 6582–6595, 2023. 1
- [14] Trevor Hastie, Andrea Montanari, Saharon Rosset, and Ryan J Tibshirani. Surprises in high-dimensional ridgeless least squares interpolation. *Annals of Statistics*, 50(2):949, 2022. 3
- [15] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European Conference on Computer Vision*, pages 709–727. Springer, 2022. 2
- [16] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 2017. 6
- [17] Zixuan Ke, Bing Liu, Nianzu Ma, Hu Xu, and Lei Shu. Achieving forgetting prevention and knowledge transfer in continual learning. *Advances in Neural Information Processing Systems*, 34:22443–22456, 2021. 1
- [18] Aditya Khosla, Nityananda Jayadevaprakash, Bangpeng Yao, and Li Fei-Fei. Novel dataset for fine-grained image categorization. In *First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition*, Colorado Springs, CO, 2011. 6
- [19] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 2
- [20] A Krizhevsky. Learning multiple layers of features from tiny images. *Master's thesis, University of Tront*, 2009. 6
- [21] Jiaqi Li, Yuanhao Lai, Rui Wang, Changjian Shui, Sabyasachi Sahoo, Charles X Ling, Shichun Yang, Boyu Wang, Christian Gagné, and Fan Zhou. Hessian aware low-rank perturbation for order-robust continual learning. *IEEE Transactions on Knowledge and Data Engineering*, 2024. 1, 2, 3, 7
- [22] Yujie Li, Xin Yang, Hao Wang, Xiangkun Wang, and Tianrui Li. Learning to prompt knowledge transfer for open-world continual learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 13700–13708, 2024. 2
- [23] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2017. 2
- [24] Dongze Lian, Daquan Zhou, Jiashi Feng, and Xinchao Wang. Scaling & shifting your features: A new baseline for efficient model tuning. *Advances in Neural Information Processing Systems*, 35:109–123, 2022. 6, 7
- [25] Sen Lin, Peizhong Ju, Yingbin Liang, and Ness Shroff. Theory on forgetting and generalization of continual learning. In *International Conference on Machine Learning*, pages 21078–21100. PMLR, 2023. 1, 3

- [26] Xialei Liu, Chenshen Wu, Mikel Menta, Luis Herranz, Bogdan Raducanu, Andrew D Bagdanov, Shangling Jui, and Joost van de Weijer. Generative feature replay for class-incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 226–227, 2020. 2
- [27] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. *Advances in Neural Information Processing Systems*, 30, 2017. 2
- [28] Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, pages 109–165. Elsevier, 1989. 1
- [29] Mark D McDonnell, Dong Gong, Amin Parvaneh, Ehsan Abbasnejad, and Anton van den Hengel. Ranpac: Random projections and pre-trained models for continual learning. *Advances in Neural Information Processing Systems*, 36, 2024. 2, 6
- [30] Leland McInnes, John Healy, Nathaniel Saul, and Lukas Großberger. Umap: Uniform manifold approximation and projection. *J. Open Source Softw*, 3:861, 2018. 8
- [31] Vidya Muthukumar, Kailas Vodrahalli, Vignesh Subramanian, and Anant Sahai. Harmless interpolation of noisy data in regression. *IEEE Journal on Selected Areas in Information Theory*, 1(1):67–83, 2020. 3
- [32] Oleksiy Ostapenko, Pau Rodriguez, Massimo Caccia, and Laurent Charlin. Continual learning via local module composition. *Advances in Neural Information Processing Systems*, 34:30298–30312, 2021. 2
- [33] Aristeidis Panos, Yuriko Kobe, Daniel Olmeda Reino, Rahaf Aljundi, and Richard E Turner. First session adaptation: A strong replay-free baseline for class-incremental learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18820–18830, 2023. 2
- [34] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019. 1
- [35] Leif E Peterson. K-nearest neighbor. *Scholarpedia*, 4(2):1883, 2009. 6
- [36] Matthew Riemer, Ignacio Cases, Robert Ajemian, Miao Liu, Irina Rish, Yuhai Tu, and Gerald Tesauro. Learning to learn without forgetting by maximizing transfer and minimizing interference. *arXiv preprint arXiv:1810.11910*, 2018. 2
- [37] Haozhe Shan, Qianyi Li, and Haim Sompolinsky. Order parameters and phase transitions of continual learning in deep neural networks. *arXiv preprint arXiv:2407.10315*, 2024. 1, 3
- [38] Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. *Advances in Neural Information Processing Systems*, 30, 2017. 2
- [39] James Seale Smith, Leonid Karlinsky, Vyshnavi Gutta, Paola Cascante-Bonilla, Donghyun Kim, Assaf Arbelle, Rameswar Panda, Rogerio Feris, and Zsolt Kira. Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11909–11919, 2023. 2, 6
- [40] Hai-Long Sun, Da-Wei Zhou, De-Chuan Zhan, and Han-Jia Ye. Pilot: A pre-trained model-based continual learning toolbox, 2025. 6
- [41] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9 (11), 2008. 8
- [42] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie. The Caltech-UCSD Birds-200-2011 Dataset. Technical Report CNS-TR-2011-001, California Institute of Technology, 2011. 6
- [43] Fu-Yun Wang, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. Foster: Feature boosting and compression for class-incremental learning. In *European Conference on Computer Vision*, pages 398–414. Springer, 2022. 2
- [44] Hao Wang, Bing Liu, Shuai Wang, Nianzu Ma, and Yan Yang. Forward and backward knowledge transfer for sentiment classification. In *Asian Conference on Machine Learning*, pages 457–472. PMLR, 2019. 1
- [45] Liyuan Wang, Xingxing Zhang, Qian Li, Jun Zhu, and Yi Zhong. Coscl: Cooperation of small continual learners is stronger than a big one. In *European Conference on Computer Vision*, pages 254–271. Springer, 2022. 2
- [46] Liyuan Wang, Jingyi Xie, Xingxing Zhang, Mingyi Huang, Hang Su, and Jun Zhu. Hierarchical decomposition of prompt-based continual learning: Rethinking obscured sub-optimality. In *Advances in Neural Information Processing Systems*, pages 69054–69076, 2023. 2
- [47] Liyuan Wang, Xingxing Zhang, Qian Li, Mingtian Zhang, Hang Su, Jun Zhu, and Yi Zhong. Incorporating neuro-inspired adaptability for continual learning in artificial intelligence. *Nature Machine Intelligence*, 5(12):1356–1368, 2023. 2
- [48] Liyuan Wang, Jingyi Xie, Xingxing Zhang, Mingyi Huang, Hang Su, and Jun Zhu. Hierarchical decomposition of prompt-based continual learning: Rethinking obscured sub-optimality. *Advances in Neural Information Processing Systems*, 36, 2024. 2
- [49] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 1, 2
- [50] Yabin Wang, Zhiheng Ma, Zhiwu Huang, Yaowei Wang, Zhou Su, and Xiaopeng Hong. Isolation and impartial aggregation: A paradigm of incremental learning without interference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 10209–10217, 2023. 2
- [51] Zhen Wang, Liu Liu, Yiqun Duan, and Dacheng Tao. Continual learning through retrieval and imagination. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8594–8602, 2022. 2
- [52] Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, et al. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *European Conference on Computer Vision*, pages 631–648. Springer, 2022. 2, 6
- [53] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer

- Dy, and Tomas Pfister. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 139–149, 2022. [2](#), [6](#)
- [54] Dominic JA Welsh and Martin B Powell. An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 10(1):85–86, 1967. [5](#), [8](#)
- [55] Tongtong Wu, Xuekai Li, Yuan-Fang Li, Gholamreza Haffari, Guilin Qi, Yujin Zhu, and Guoqiang Xu. Curriculum-meta learning for order-robust continual relation extraction. In *Proceedings of the AAAI conference on artificial intelligence*, pages 10363–10369, 2021. [3](#)
- [56] Han-Jia Ye, De-Chuan Zhan, Xue-Min Si, and Yuan Jiang. Learning mahalanobis distance metric: Considering instance disturbance helps. In *IJCAI*, pages 3315–3321, 2017. [2](#)
- [57] Han-Jia Ye, De-Chuan Zhan, Yuan Jiang, and Zhi-Hua Zhou. Heterogeneous few-shot model rectification with semantic mapping. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3878–3891, 2020. [1](#)
- [58] Jaehong Yoon, Saehoon Kim, Eunho Yang, and Sung Ju Hwang. Scalable and order-robust continual learning with additive parameter decomposition. *arXiv preprint arXiv:1902.09432*, 2019. [1](#), [2](#)
- [59] Yuanhan Zhang, Zhenfei Yin, Jing Shao, and Ziwei Liu. Benchmarking omni-vision representation through the lens of visual realms. In *European Conference on Computer Vision*, pages 594–611. Springer, 2022. [6](#)
- [60] Hanbin Zhao, Yongjian Fu, Mintong Kang, Qi Tian, Fei Wu, and Xi Li. Mgsvf: Multi-grained slow versus fast framework for few-shot class-incremental learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(3):1576–1588, 2021. [1](#)
- [61] Zangwei Zheng, Mingyuan Ma, Kai Wang, Ziheng Qin, Xiangyu Yue, and Yang You. Preventing zero-shot transfer degradation in continual learning of vision-language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19125–19136, 2023. [2](#)
- [62] Da-Wei Zhou, Zi-Wen Cai, Han-Jia Ye, De-Chuan Zhan, and Ziwei Liu. Revisiting class-incremental learning with pre-trained models: Generalizability and adaptivity are all you need. *International Journal of Computer Vision*, pages 1–21, 2024. [2](#), [6](#)
- [63] Da-Wei Zhou, Hai-Long Sun, Jingyi Ning, Han-Jia Ye, and De-Chuan Zhan. Continual learning with pre-trained models: a survey. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 8363–8371, 2024. [2](#)
- [64] Da-Wei Zhou, Hai-Long Sun, Han-Jia Ye, and De-Chuan Zhan. Expandable subspace ensemble for pre-trained model-based class-incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23554–23564, 2024. [2](#), [6](#)
- [65] Da-Wei Zhou, Yuanhan Zhang, Yan Wang, Jingyi Ning, Han-Jia Ye, De-Chuan Zhan, and Ziwei Liu. Learning without forgetting for vision-language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. [2](#), [6](#)
- [66] Kai Zhu, Wei Zhai, Yang Cao, Jiebo Luo, and Zheng-Jun Zha. Self-sustaining representation expansion for non-exemplar class-incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9296–9305, 2022. [2](#)